

UNIT – 2

LEETCODE EXERCISE

1. Find Users with Valid E-Mails

Problem Summary

You are given a table named **Users**.

Column Name	Type
user_id	int
Name	varchar
mail	varchar(50)

- user_id is the primary key.
- Each row contains the ID and name (which is an e-mail address) of a user. You need

to find users whose e-mail addresses are **valid**.

Valid E-mail Rules

A valid e-mail:

1. Starts with a letter (a-z or A-Z)
2. Followed by:
 - letters (a-z, A-Z)
 - digits (0-9)
 - underscore (_)
 - period (.)
 - hyphen (-)
3. Ends with:
@leetcode.com

SQL Schema:

```
Create table If Not Exists Users (user_id int, name varchar(30), mail varchar(50)); Truncate table Users;
insert into Users (user_id, name, mail) values ('1', 'Winston', 'winston@leetcode.com'); insert into Users (user_id, name, mail) values ('2', 'Jonathan', 'jonathanisgreat');
insert into Users (user_id, name, mail) values ('3', 'Annabelle', 'bella-@leetcode.com'); insert into Users (user_id, name, mail) values ('4', 'Sally', 'sally.come@leetcode.com');
insert into Users (user_id, name, mail) values ('5', 'Marwan', 'quarz#2020@leetcode.com'); insert into Users (user_id, name, mail) values ('6', 'David', 'david69@gmail.com');
insert into Users (user_id, name, mail) values ('7', 'Shapiro', 'shapo@leetcode.com');
```

Users

user_id	name	mail
1	Winston	winston@leetcode.com
2	Jonathan	jonathanisgreat
3	Annabelle	annabelle@leetcode.com
4	Sally	sally.com
5	Marwan	marwan@leetcode
6	David	david99@leetcode.com
7	Shapiro	.shapiro@leetcode.com

Task

Write an SQL query to find all users having **valid e-mail addresses**.

Return the result table in any order.

MySQL Solution

```
</> Code
MySQL v Auto
1 # Write your MySQL query statement below
2 SELECT *
3 FROM Users
4 WHERE REGEXP_LIKE(mail, '^[A-Za-z][A-Za-z0-9._-]*@leetcode\\.com$', 'c');
```

Saved Ln 4, Col 74

Testcase Test Result

Accepted Runtime: 65 ms

Case 1

Input

Users =

user_id	name	mail
1	Winston	winston@leetcode.com
2	Jonathan	jonathanisgreat
3	Annabelle	bella@leetcode.com
4	Sally	sally.come@leetcode.com
5	Marwan	quarz#2020@leetcode.com
6	David	david69@gmail.com
7	Shapiro	.shapo@leetcode.com

View less

Output

user_id	name	mail
1	Winston	winston@leetcode.com
3	Annabelle	bella@leetcode.com
4	Sally	sally.come@leetcode.com

2. Patients With a Condition

Problem Summary

You are given a table named **Patients**.

Column Name	Type
patient_id	int
patient_name	varchar
conditions	varchar

- patient_id is the primary key.

Important Rule

A patient is considered valid if:

- DIAB1 appears **at the beginning** of the string, or
- DIAB1 appears **after a space**.

Table Script:

```
Create table If Not Exists Patients (patient_id int, patient_name varchar(30), conditions varchar(100));
Truncate table Patients;
insert into Patients (patient_id, patient_name, conditions) values ('1', 'Daniel', 'YFEV COUGH');
insert into Patients (patient_id, patient_name, conditions) values ('2', 'Alice', '');
insert into Patients (patient_id, patient_name, conditions) values ('3', 'Bob', 'DIAB100 MYOP');
insert into Patients (patient_id, patient_name, conditions) values ('4', 'George', 'ACNE DIAB100');
insert into Patients (patient_id, patient_name, conditions) values ('5', 'Alain', 'DIAB201');
```

Input

Patients

patient_id	patient_name	conditions
1	Daniel	YFEV COUGH
2	Alice	
3	Bob	DIAB100 MYOP
4	George	ACNE DIAB100
5	Alain	DIAB201

Task

Write an SQL query to report the patient ID, patient name, and conditions of patients who have **Type I Diabetes (DIAB1)**.

Return the result table in any order.

MySQL Solution

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT patient_id, patient_name, conditions
3 FROM Patients
4 WHERE conditions LIKE 'DIAB1%'
5    OR conditions LIKE '% DIAB1%';
```

Testcase Test Result

Accepted Runtime: 67 ms

Case 1

Input

Patients =

patient_id	patient_name	conditions
1	Daniel	YFEV COUGH
2	Alice	
3	Bob	DIAB100 MYOP
4	George	ACNE DIAB100
5	Alain	DIAB201

Output

patient_id	patient_name	conditions
3	Bob	DIAB100 MYOP
4	George	ACNE DIAB100

3. Second Highest Salary

Problem Summary

You are given an **Employee** table.

Column Name	Type
id	int
salary	int

- id is the primary key.
- Each row contains the salary of an employee.
- Salaries may contain **duplicates**.

Your task is to find the **second highest distinct salary**.
If there is no second highest salary, return NULL.

Table Script:

```
Create table If Not Exists Employee (id int, salary int);
Truncate table Employee;
insert into Employee (id, salary) values ('1', '100');
insert into Employee (id, salary) values ('2', '200');
insert into Employee (id, salary) values ('3', '300');
```

Input

Employee

id	salary
1	100
2	200
3	300

Explanation

Distinct salaries:

100, 200, 300

Highest salary = 300

Second highest salary = 200

Task

Write an SQL query to report the **second highest distinct salary** from the Employee table.

MySQL Solution

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT MAX(salary) AS SecondHighestSalary
3 FROM Employee
4 WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Saved

Ln 4, Col 51

Testcase Test Result

Accepted Runtime: 104 ms

Case 1 Case 2

Input

Employee =

id	salary
1	100
2	200
3	300

Output

SecondHighestSalary
200

4. Customer Placing the Largest Number of Orders

Problem Summary

You are given an **Orders** table.

Column Name	Type
order_number	int
customer_number	int

- order_number is unique.
- Each row represents one order placed by a customer.
- A customer can place multiple orders.

Your task is to find the **customer who placed the largest number of orders**.

The problem guarantees that:

There is exactly **one customer** who has placed the maximum number of orders.

Table Script:

Create table If Not Exists orders (order_number int, customer_number int);

Truncate table orders;

insert into orders (order_number, customer_number) values ('1', '1');

insert into orders (order_number, customer_number) values ('2', '2');

insert into orders (order_number, customer_number) values ('3', '3');

insert into orders (order_number, customer_number) values ('4', '3');

Input

Orders

order_number	customer_number
1	1
2	2
3	3
4	3

Task

Write an SQL query to find the **customer_number** of the customer who has placed the largest number of orders.

Return the result table in any order.

MySQL Solution

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT customer_number
3 FROM Orders
4 GROUP BY customer_number
5 ORDER BY COUNT(order_number) DESC
6 LIMIT 1;
```

Saved

Ln 6, Col 9

Testcase Test Result

Accepted Runtime: 72 ms

Case 1

Input

orders =

order_number	customer_number
1	1
2	2
3	3
4	3

Output

customer_number
3

5. Product Sales Analysis III

Problem Summary

You are given two tables, but only the **Sales** table is needed for this problem.

Sales Table

Column Name	Type
sale_id	int
product_id	int
year	int
quantity	int
price	int

- (sale_id, year) is the primary key.
- Each row represents a sale of a product in a particular year.

Objective

For **each product**, find:

1. The **first year** in which the product was sold.
2. The **quantity** sold in that year.
3. The **price** of the product in that year.

Table Script:

```
Create table If Not Exists Sales (sale_id int, product_id int, year int, quantity int, price int);
Truncate table Sales;
insert into Sales (sale_id, product_id, year, quantity, price) values ('1', '100', '2008', '10', '5000');
insert into Sales (sale_id, product_id, year, quantity, price) values ('2', '100', '2009', '12', '5000');
insert into Sales (sale_id, product_id, year, quantity, price) values ('7', '200', '2011', '15', '9000');
```

Sales

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Task

Write an SQL query to report:

- product_id
- first_year
- quantity
- price

for the **first year** in which each product was sold.

Return the result table in any order.

MySQL Solution

</> Code

MySQL Auto

```
1 # Write your MySQL query statement below
2 SELECT
3     product_id,
4     year AS first_year,
5     quantity,
6     price
7 FROM Sales
8 WHERE (product_id, year) IN (
9     SELECT product_id, MIN(year)
10    FROM Sales
11   GROUP BY product_id
12 );
```

Saved

Ln 12, Col 3

Testcase

Test Result

Accepted Runtime: 101 ms

Case 1

Input

Sales =

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Output

product_id	first_year	quantity	price
100	2008	10	5000
200	2011	15	9000